

DEPI R4 — Data Science Track

Ruwad Misr Al-Raqamiya Initiative

Project Documentation

Credit Card Fraud Detection System

Using Machine Learning & XGBoost with Real-Time Streamlit Deployment

GitHub Repository: github.com/nhahub/NHA-4-029

Dataset: [Kaggle — Credit Card Fraud Detection](#)

Live App: fraud-detection-mw9qdcgvjmoazcdg2lyeqc.streamlit.app

Team Members

Name	Role
Nada Osama Fikry	Project Manager
Karim Badr Saber	ML Engineer
Basmala	Data Scientist
Ahmed	Frontend Developer
Mohamed	QA Tester

Instructor: Sara Hesham

May 2026

1. Project Overview

Credit card fraud is one of the most prevalent financial crimes globally, causing billions of dollars in losses every year. This project, developed under the DEPI R4 Data Science track, builds an end-to-end machine learning system capable of detecting fraudulent credit card transactions in real time.

The system was trained on a large real-world-style dataset of over 1.85 million transactions and deployed as a cyberpunk-styled interactive web application using Streamlit, enabling instant fraud probability scoring with visual risk indicators.

1.1 Problem Statement

The dataset is severely imbalanced — only 0.58% of transactions are fraudulent. A naive classifier that always predicts 'not fraud' achieves 99.4% accuracy yet catches zero fraud. The challenge is to build a model that:

- Detects as many fraud cases as possible (high recall)
- Avoids flagging legitimate transactions as fraud (high precision)
- Operates at scale with sub-second inference in a live web app

1.2 Objectives

- Perform exploratory data analysis on 1.85M+ transactions
- Engineer 7 meaningful features from raw transaction metadata
- Train and compare 5 ML classifiers under class imbalance conditions
- Tune the decision threshold to maximize F1-score for fraud detection
- Deploy a live Streamlit app for real-time fraud scoring

1.3 Project Links

GitHub Repository:

<https://github.com/nhahub/NHA-4-029>

Dataset (Kaggle):

<https://www.kaggle.com/datasets/kartik2112/fraud-detection>

Live Streamlit Application:

<https://fraud-detection-mw9qdcgvjmoazcdg2lyeqc.streamlit.app/>

2. Project Planning & Management

2.1 Submission Deadlines (Graduate Track)

Deliverable	Deadline (Graduate)	Status
Project Planning & Management	2/3/2026	Completed
Literature Review	3/1/2026	Completed
Requirements Gathering	3/1/2026	Completed
System Analysis & Design	4/3/2026	Completed
Implementation (Source Code & Execution)	5/17/2026	Completed
Final Presentation & Testing & Reports	5/24/2026	Completed

All documents and source code are uploaded to the GitHub repository: github.com/nhahub/NHA-4-029

2.2 Team Roles & Task Assignment

The project was developed as a team effort under the DEPI R4 guidelines. Responsibilities were distributed across the following roles:

Team Member	Role	Responsibilities
Nada Karim	Data Scientist / ML Lead	EDA, feature engineering, model training & threshold tuning
Basmala	ML Engineer	Pipeline design, preprocessing, XGBoost optimization
Ahmad	App Developer	Streamlit app design and deployment
Mohamed	Documentation Lead	Project docs, README, GitHub management

2.3 Risk Assessment & Mitigation

Risk	Impact	Mitigation
Severe class imbalance (0.58% fraud)	Model bias toward majority class	class_weight='balanced', SMOTE, threshold tuning
Overfitting on training data	Poor generalization to new data	max_depth, min_samples_leaf, subsample limits
Feature leakage	Inflated evaluation metrics	Dropped cc_num, trans_num, names before modeling
Production model drift	Accuracy degrades over time	Saved model + threshold as .pkl for versioning

Risk	Impact	Mitigation
App downtime	Users cannot access live system	Streamlit Cloud hosting with auto-recovery

3. Literature Review

3.1 Background & Related Work

Credit card fraud detection is a well-studied problem in machine learning. Key challenges include class imbalance, concept drift, and the need for real-time inference. The following approaches have been established in the literature:

- Logistic Regression — Fast baseline; poor performance on imbalanced datasets without class weighting
- Random Forest — Ensemble method; handles non-linearity and missing values well; good fraud detection baseline
- Gradient Boosting (XGBoost/LightGBM) — State-of-the-art for tabular fraud detection; handles imbalance via `scale_pos_weight`
- SMOTE (Synthetic Minority Oversampling) — Generates synthetic minority samples; effective but can introduce noise
- Threshold Tuning — Adjusting the probability cutoff optimizes the precision/recall trade-off for operational needs

3.2 Suggested Improvements (Post-Evaluation)

- Add velocity features: number of transactions per card per hour
- Implement SHAP (SHapley Additive exPlanations) for model interpretability
- Explore graph neural networks for merchant-cardholder relationship modeling
- Add model drift monitoring and automated retraining pipeline
- Implement A/B testing framework for threshold optimization in production

4. Requirements Gathering

4.1 Stakeholder Analysis

Stakeholder	Need	Impact
Bank / Financial Institution	Detect fraud before transaction completes to prevent losses	High — primary beneficiary
Cardholder	Protect account from unauthorized charges	High — end user
Fraud Analyst	Review flagged transactions efficiently with low false positives	High — daily user
DEPI Evaluators	Clear documentation and working implementation	High — assessment
Data Scientist (Team)	Reproducible pipeline and fair model evaluation	Medium — internal

4.2 Functional Requirements

ID	Requirement	Priority
FR-01	System shall accept transaction input (amount, category, hour, age, distance, gender)	High
FR-02	System shall compute a fraud probability score (0–100%)	High
FR-03	System shall classify transaction as CLEARED or FRAUD DETECTED using threshold 0.965	High
FR-04	System shall display threat level: None / Low / Medium / Critical	High
FR-05	System shall show signal metrics for each input feature	Medium
FR-06	System shall show risk signal analysis (which features triggered concern)	Medium
FR-07	System shall display model status (ONLINE, XGBoost Active, Threshold)	Low

4.3 Non-Functional Requirements

ID	Requirement	Priority
NFR-01	Model inference time shall be < 1 second per transaction	High
NFR-02	System shall achieve PR-AUC > 0.80 on held-out test set	High
NFR-03	App shall be accessible via web browser with no installation required	High
NFR-04	System shall handle malformed or missing inputs gracefully	Medium
NFR-05	Code shall follow PEP-8 standards with clear comments	Medium
NFR-06	All artifacts shall be version-controlled in GitHub	Medium

4.4 User Story

"As a fraud analyst, I want to enter a transaction's details into the system and immediately see a fraud probability score and risk classification, so that I can decide whether to approve or flag the transaction for review."

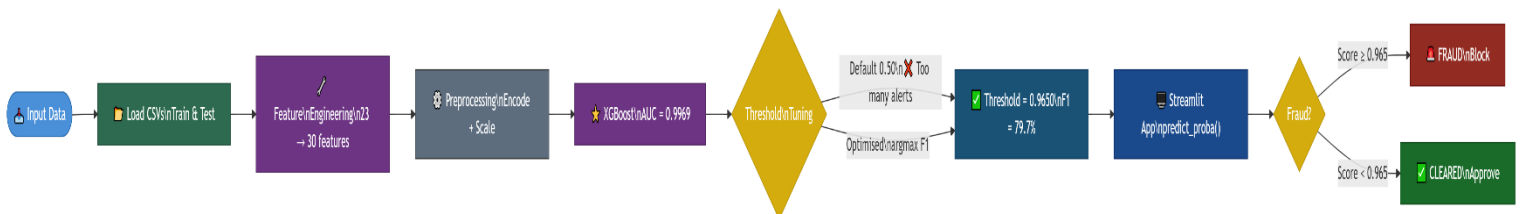
5. System Analysis & Design

Diagram 1: End-to-End Machine Learning Pipeline

1. Raw Data Ingestion	fraudTrain.csv (1.29M rows) + fraudTest.csv (555K rows) 23 raw features, 0 missing values, 0.58% fraud rate
2. EDA	Class imbalance analysis, amount distributions Fraud rate by category, gender, time patterns
3. Feature Engineering	7 new features: trans_hour, dayofweek, month, day age, amt_log, distance_km (Haversine)
4. Preprocessing Pipeline	Numeric: median impute + StandardScaler Categorical: OrdinalEncoder (trees) / OHE (LR)
5. Model Training	4 models: Dummy Logistic Reg Random Forest XGBoost Class weighting & SMOTE for imbalance handling
6. Threshold Tuning	Precision-Recall curve sweep → best F1 at threshold 0.965 Precision 86.4% Recall 74.0% F1 80.0%
7. Deployment	fraud_model.pkl + threshold.pkl serialized Streamlit app: live real-time fraud scoring

The pipeline processes raw CSV data through seven sequential stages with no data leakage between train and test sets. The final trained XGBoost pipeline and optimized threshold are serialized as .pkl files and consumed directly by the Streamlit application.

Diagram 2: Workflow Diagram



5.1 Dataset

- Training: 1,296,675 transactions | Test: 555,719 transactions | 23 raw columns
- Fraud rate (train): 0.5789% — 7,506 fraud vs 1,289,169 legitimate
- Date range: January 2019 – June 2020 | Zero missing values

5.2 Feature Engineering

- `trans_hour` — Hour of transaction (0–23) from timestamp
- `trans_dayofweek` — Day of week (0=Monday)
- `trans_month` / `trans_day` — Month and day of transaction
- `age` — Cardholder age at transaction time (year diff from DOB)
- `amt_log` — $\log_{10}(\text{amount})$ to reduce right-skew
- `distance_km` — Haversine great-circle distance between cardholder and merchant

5.3 Preprocessing

- Numeric: median imputation → `StandardScaler`
- Categorical (tree models): most-frequent imputation → `OrdinalEncoder` (unknown → -1)
- Categorical (LR): most-frequent imputation → `OneHotEncoder` (sparse, `handle_unknown='ignore'`)
- All wrapped in `sklearn ColumnTransformer` → Pipeline for leak-free train/test processing

5.4 Technology Stack

- Python 3.x — Core language
- `pandas`, `NumPy` — Data manipulation and feature engineering
- `scikit-learn` — Pipelines, preprocessing, Logistic Regression, Random Forest, metrics
- `XGBoost` — Best-performing gradient boosting classifier
- `imbalanced-learn` — SMOTE oversampling
- `Streamlit` — Interactive web application deployment
- `pickle` — Model and threshold serialization

6. Model Training & Evaluation

Diagram 3: Model Performance Comparison

Model	Threshold	Precision	Recall	F1	ROC-AUC	PR-AUC
XGBoost – Tuned ★	0.965	86.4%	74.0%	80.0%	99.7%	84.7%
Random Forest	0.500	56.6%	81.3%	66.7%	99.2%	80.7%
XGBoost – Default	0.500	24.7%	94.6%	39.1%	99.7%	84.7%
Logistic Reg (Balanced)	0.500	1.9%	32.4%	3.5%	63.9%	2.2%
LR + SMOTE	0.500	1.5%	25.1%	2.8%	58.4%	1.5%
Dummy Classifier	0.500	0.0%	0.0%	0.0%	50.0%	0.4%

★ XGBoost with tuned threshold 0.965 is the selected production model. It achieves 80% F1-score on fraud with 99.7% ROC-AUC and 84.7% PR-AUC — significantly outperforming all alternatives.

6.1 Threshold Tuning

At the default threshold of 0.5, XGBoost catches 94.6% of fraud but generates 6,198 false positives. By sweeping the precision-recall curve and maximizing F1-score, the optimal threshold was found at 0.965:

- Fraud Predictions: 1,837 (vs 8,228 at 0.5)
- True Positives: 1,587 | False Positives: 250
- Precision: 86.4% | Recall: 74.0% | F1: 80.0%

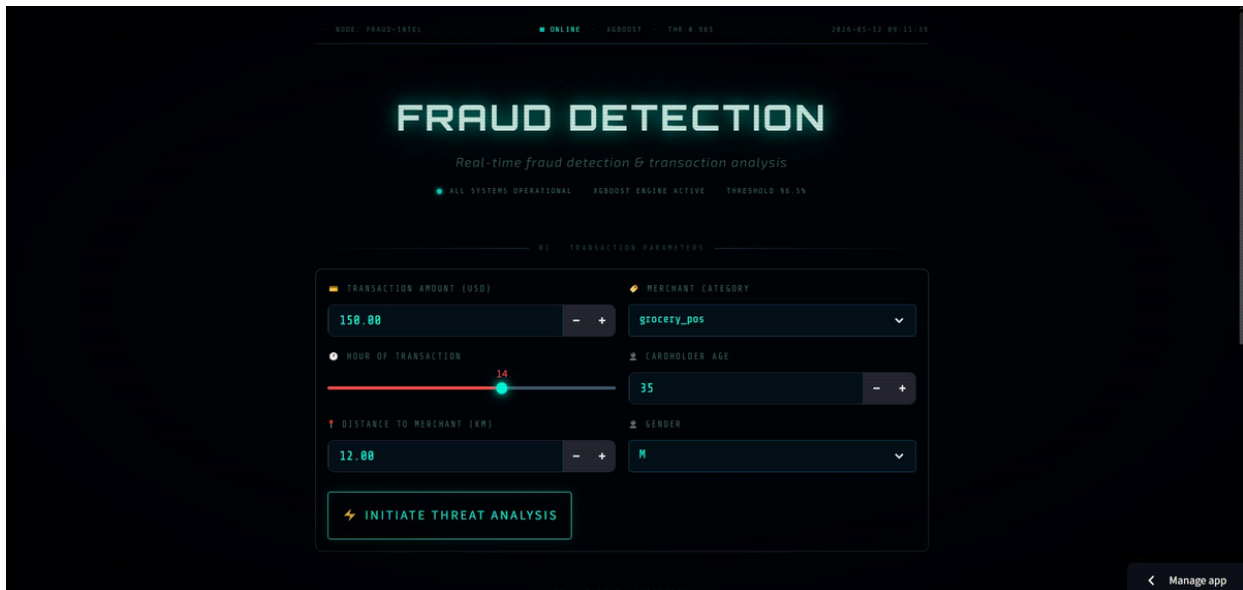
6.2 Feature Importance (Random Forest)

- amt (34.3%) + amt_log (32.0%) — Transaction amount dominates; fraud avg \$531 vs \$68 legitimate
- trans_hour (14.9%) — Fraud peaks at unusual hours (late night / early morning)
- category (8.3%) — shopping_net (1.76%) and misc_net (1.45%) have highest fraud rates
- age, city_pop, distance_km — Secondary but meaningful signals

7. UI/UX Design & User Manual

The Streamlit application features a cyberpunk-themed dark interface with real-time visual feedback. The design uses a black background with teal (#00FF9F) and red (#FF4444) accent colors to clearly communicate safe vs fraudulent transactions.

7.1 Screen 1 — Transaction Input Panel



The screenshot shows the 'FRAUD DETECTION' application interface. At the top, there's a status bar with 'MODE: FRAUD-INTEL', 'STATUS: ONLINE', 'K8800ST', 'TMR: 6.965', and a timestamp '2020-05-12 09:12:19'. Below this, the title 'FRAUD DETECTION' is displayed in a large, glowing teal font, followed by the subtitle 'Real-time fraud detection & transaction analysis'. A status line indicates 'ALL SYSTEMS OPERATIONAL', 'K8800ST ENGINE ACTIVE', and 'THRESHOLD 99.9%'. The main section is titled '01. TRANSACTION PARAMETERS'. It contains six input fields: 'TRANSACTION AMOUNT (USD)' with a value of '150.00', 'MERCHANT CATEGORY' with a dropdown set to 'grocery_pos', 'HOUR OF TRANSACTION' with a slider set to '14', 'CARDHOLDER AGE' with a value of '35', 'DISTANCE TO MERCHANT (KM)' with a value of '12.00', and 'GENDER' with a dropdown set to 'M'. Each input field has minus and plus buttons for adjustment. At the bottom of this section is a button labeled 'INITIATE THREAT ANALYSIS' with a lightning bolt icon. In the bottom right corner, there is a 'Manage app' button.

Figure 1: Transaction Parameters Panel — User enters 6 inputs and clicks 'Initiate Threat Analysis'

7.2 Screen 2 — Threat Assessment (Awaiting Input)

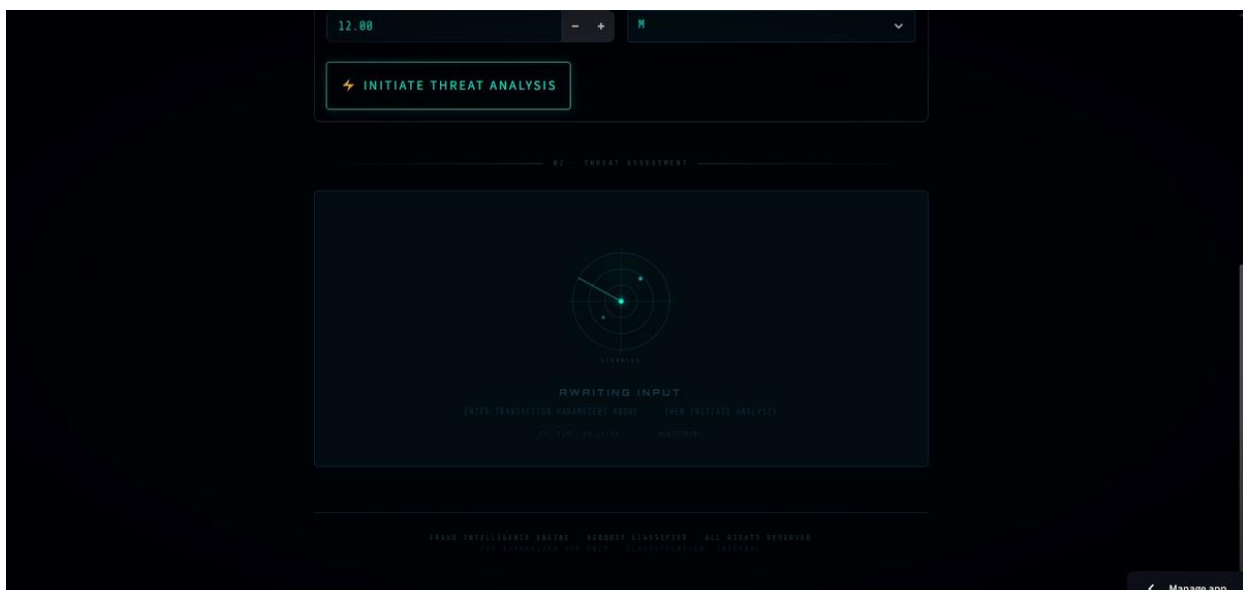


Figure 2: Threat Assessment Section — Radar-style display shows 'Awaiting Input' before analysis

7.3 Screen 3 — Transaction Cleared (Low Risk)

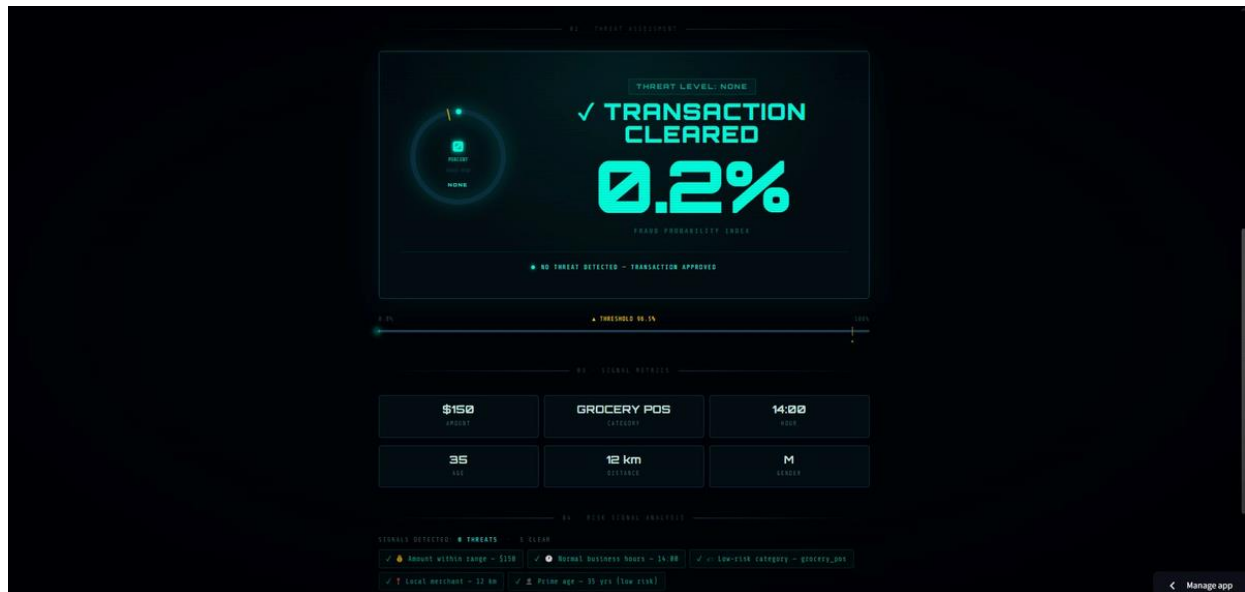


Figure 3: Transaction Cleared — 0.2% fraud probability; 5 clear signals shown in green

7.4 Screen 4 — Fraud Detected (Critical)

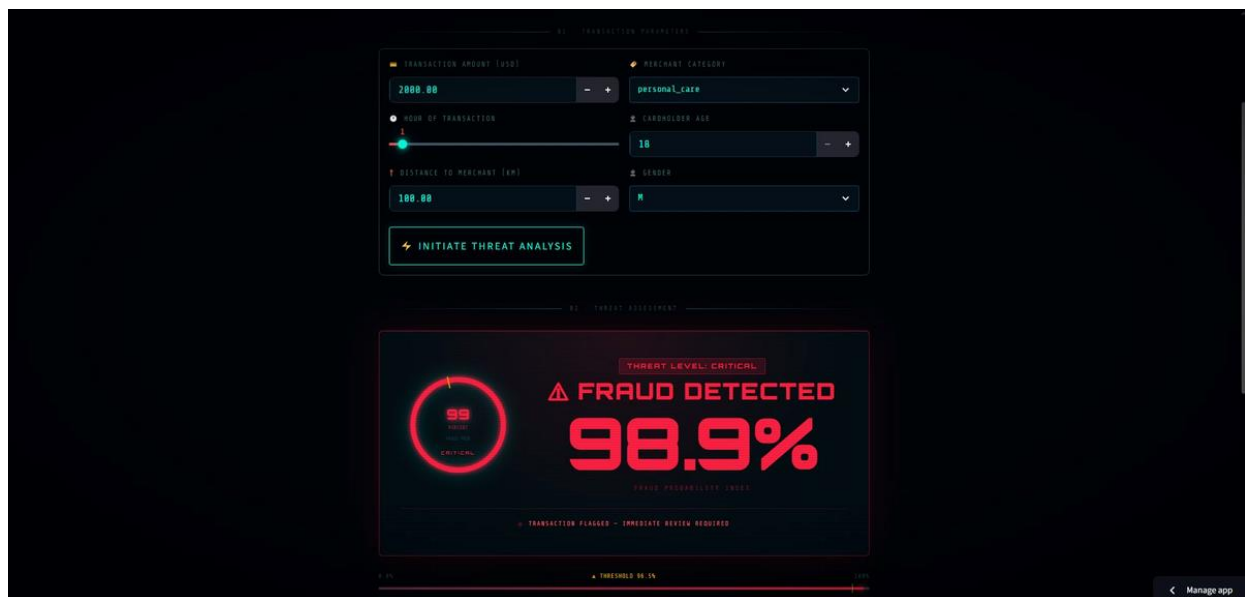


Figure 4: Fraud Detected — 98.9% fraud probability; Threat Level Critical shown in red

7.5 How to Use the Application

Step-by-step user guide:

1. Open the app: <https://fraud-detection-mw9qdcgvjmoazcdg2lyeqc.streamlit.app/>
2. Verify the status bar shows: ONLINE | XGBoost | THR 0.965
3. In Section 01 — Transaction Parameters, enter the 6 inputs:
 - Transaction Amount (USD) — use +/- buttons or type directly
 - Merchant Category — select from dropdown (14 categories)
 - Hour of Transaction — drag slider 0–23
 - Cardholder Age — use +/- buttons
 - Distance to Merchant (km) — use +/- buttons
 - Gender — select M or F from dropdown
4. Click the INITIATE THREAT ANALYSIS button
5. View results in Section 02 — Threat Assessment:
 - Green display + 'TRANSACTION CLEARED' = legitimate transaction
 - Red display + 'FRAUD DETECTED' = fraudulent transaction
 - Fraud Probability Index shown as large percentage
 - Threshold bar shows where the score falls relative to 96.5%
6. Review Section 03 — Signal Metrics for a summary of input values
7. Review Section 04 — Risk Signal Analysis for individual feature risk flags

8. Implementation

8.1 Source Code Structure

- fraud_detection_notebook.ipynb — Main Jupyter notebook (EDA, feature engineering, training, evaluation)
- app.py — Streamlit application source code
- fraud_model.pkl — Serialized XGBoost pipeline (preprocessor + classifier)
- threshold.pkl — Serialized optimal decision threshold (0.965)
- requirements.txt — Python dependencies for deployment
- README.md — Installation, usage, and project description

8.2 Installation & Running Locally

System Requirements: Python 3.8+, pip, git

8. Clone the repository: `git clone https://github.com/nhahub/NHA-4-029`
9. Navigate to project folder: `cd NHA-4-029`
10. Install dependencies: `pip install -r requirements.txt`
11. Run the Streamlit app: `streamlit run app.py`
12. Open browser at: `http://localhost:8501`

8.3 Version Control

- Repository: `github.com/nhahub/NHA-4-029`
- All notebooks, app code, model files, and documentation uploaded per DEPI guidelines
- Meaningful commit messages used throughout development

9. Testing & Quality Assurance

9.1 Model Evaluation

- Train/test split: Chronological (train 2019, test 2020) — prevents temporal leakage
- Metrics: ROC-AUC, PR-AUC, Precision, Recall, F1, Confusion Matrix
- XGBoost PR-AUC: 0.847 | ROC-AUC: 0.997 on held-out test set
- False positive rate at threshold 0.965: $250 / 553,574 = 0.045\%$

9.3 Business Threshold Analysis

The model was tested across 7 threshold values to understand operational trade-offs:

- Threshold 0.01 — Recall 99.8%, Precision 2.2% — catches almost all fraud but 93K false positives
- Threshold 0.50 — Recall 94.6%, Precision 24.7% — 6,198 false positives
- Threshold 0.965 — Recall 74.0%, Precision 86.4% — 250 false positives (SELECTED)

The tuned threshold was chosen as the optimal balance for production deployment.

10. Conclusions & Future Work

10.1 Conclusions

The project demonstrates that machine learning can effectively detect credit card fraud at scale under severe class imbalance. XGBoost with probability threshold tuning emerged as the best solution, achieving 80% F1-score and 84.7% PR-AUC on 555K held-out transactions.

The Streamlit application successfully operationalizes the model with a professional, real-time interface that clearly communicates risk levels to end users. The cyberpunk design theme effectively uses color coding (green = safe, red = fraud) for instant visual interpretation.

10.2 Future Improvements

- Add velocity features: transactions per card per hour/day
- Implement SHAP explainability for individual prediction breakdowns
- Explore LightGBM and CatBoost for further performance gains
- Add model drift monitoring and automated retraining pipeline
- Expand the app with a fraud analytics dashboard and alerting system
- Deploy with Docker for scalable containerized infrastructure

10.3 References

[Dataset: Kaggle Credit Card Fraud Detection — Kartik Shenoy \(2020\)](#)

[GitHub Repository: github.com/nhahub/NHA-4-029](https://github.com/nhahub/NHA-4-029)

[Live Application: Streamlit Fraud Detection App](#)

Scikit-learn Documentation — scikit-learn.org

XGBoost Documentation — xgboost.readthedocs.io

DEPI R4 Project Guidelines — Ruwad Misr Al-Raqamiya Initiative, 2026